Shortest answer, If I try to update a row that does not exist in python sqlite3, what exception?

No exception is raised. `cursor.execute("UPDATE ...")` succeeds with `cursor.rowcount == 0`.

Shortest answer, sqlite3 update

`cursor.execute("UPDATE t SET c=? WHERE id=?", (v, i))` then `conn.commit()`. Check `cursor.rowcount` for affected rows (0 if none).

How to make a thread wait for another to finish a sqlite3 transaction?

Use a `threading.Lock` (or `Event`) shared between threads:

```python
lock = threading.Lock()

# Thread A
with lock:
    conn.execute("BEGIN")
    conn.execute("UPDATE ...")
    conn.commit()

# Thread B
with lock:
    conn.execute("UPDATE ...")  # waits until A releases
```

Or share one connection with `check_same_thread=False` and let SQLite's own locking + a `threading.Lock` serialize access. For "wait until done" specifically, a `threading.Event`: A calls `event.set()` after `commit()`, B calls `event.wait()`.

The problem is that I'm unit testing the thread and I can't pass a lock

Use `unittest.mock` to patch the lock, or better: have the production code acquire the lock from an injectable module-level object.

Options:

1. **Patch at test time:**

```
python

with mock.patch("mymodule.lock") as m:
    ...  # m.__enter__.return_value is a MagicMock; won't actually block
```

2. **Inject via constructor/param** (cleanest): pass `lock=None` defaulting to a module-level `threading.Lock()`, so tests pass a real `threading.Lock()` or `threading.Event` they control.
3. **Use `threading.Event` in prod code** for the "wait for commit" signal — tests can set/assert on it directly without a lock:

```
python

committed = threading.Event()   # module-level
# worker:
conn.commit()
committed.set()
# test:
committed.wait(timeout=5)
```

4. **Patch `sqlite3.Connection.commit`** to block on a test-controlled `Event`, simulating the in-progress transaction:

```
python

gate = threading.Event()
with mock.patch.object(sqlite3.Connection, "commit", side_effect=lambda self: gate.wai
t()):
    ...
```

Option 2 or 3 is usually the right fix — make the synchronization primitive injectable rather than reaching for patches.

journal_mode=EXCLUSIVE?

Yes — `PRAGMA journal_mode=EXCLUSIVE` is a legitimate way to make a second thread *block* (or fail) while another holds the write lock, without any Python-level lock:

```
python
```

```
conn.execute("PRAGMA journal_mode=EXCLUSIVE")
```

With EXCLUSIVE, the locking mode prevents other connections from reading/writing while a write transaction is active. A second connection's write will **block until** `busy_timeout` **expires**, then raise:

```
text

sqlite3.OperationalError: database is locked
```

For unit testing a wait, set a `busy_timeout` and assert the second thread waits:

```
python

conn.execute("PRAGMA busy_timeout=5000")
```

Caveats:

- EXCLUSIVE locks the whole database file, not per-row — fine for a test, heavy for production.
- It's per-connection; the *first* connection that sets it holds the exclusive lock once it starts a transaction.
- The second thread must use a **separate connection** (`check_same_thread` doesn't matter then).
- If you just want deterministic blocking in a test, `busy_timeout` + a held `BEGIN IMMEDIATE` is simpler than EXCLUSIVE and behaves the same for your assertion.

So: yes, it works, but `BEGIN IMMEDIATE + busy_timeout` gives the same "wait then error" behavior with less blast radius.